

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО

ITMO University

Лабораторная работа 6

По дисциплине Администрирование платформ на ОС Linux

Тема работы Управление сервисами systemd, организация резервного копирования и автоматизация хранения данных в S3-хранилище

Обучающийся Рекун Ирина Данииловна

Факультет прикладной информатики

Группа К3220

Направление подготовки 11.03.02 Инфокоммуникационные технологии и системы связи

Образовательная программа Программирование в инфокоммуникационных системах

Обучающийся

(дата)

(подпись)

Рекун И.Д.

(Ф.И.О.)

Руководитель

(дата)

(подпись)

Береснев А.Д.

(Ф.И.О.)

ВВЕДЕНИЕ

Цель работы: получить практические навыки по настройке многоконтейнерного приложения в среде Docker Compose, организации резервного копирования данных и автоматизации хранения резервных копий в S3-хранилище.

Задачи: в ходе выполнения лабораторной работы необходимо:

1. Подготовить структуру проекта для размещения конфигурационных файлов и скриптов;
2. Создать конфигурационный файл docker-compose.yml для запуска нескольких сервисов;
3. Настроить веб-сервер Nginx в качестве обратного прокси-сервера;
4. Организовать балансировку нагрузки между несколькими backend-сервисами;
5. Подготовить Dockerfile-файлы для сервисов проекта;
6. Настроить контейнер с базой данных PostgreSQL;
7. Проверить корректность конфигурации Docker Compose;
8. Запустить многоконтейнерную инфраструктуру с помощью Docker Compose;
9. Настроить сохранение данных и логов с использованием volumes;
10. Организовать резервное копирование базы данных и логов;
11. Настроить передачу резервных копий в S3-хранилище;
12. Проверить работоспособность созданной инфраструктуры и автоматизированных скриптов.

На рисунке 1 показан процесс создания структуры проекта. Были созданы каталоги для сервисов nginx, libspeed, postgres, backup-db и backup-logs, а также подготовлены основные файлы конфигурации. После этого в соответствующие каталоги были скопированы конфигурация Nginx и файлы приложения Librespeed.

```
root@rekunid:~# mkdir -p ~/project/{nginx,librespeed/app,postgres,backup-db,backup-logs}
root@rekunid:~# cd ~/project
root@rekunid:~/project# touch docker-compose.yml .env
root@rekunid:~/project# touch nginx/Dockerfile nginx/nginx.conf
root@rekunid:~/project# touch librespeed/Dockerfile
root@rekunid:~/project# touch postgres/Dockerfile postgres/init.sql
root@rekunid:~/project# touch backup-db/Dockerfile backup-db/backup.sh
root@rekunid:~/project# touch backup-logs/Dockerfile backup-logs/backup-nginx-logs.sh
```

Рисунок 1 — Создание структуры проекта и подготовка файлов для Docker Compose

После создания структуры проекта были перенесены необходимые файлы для работы сервисов. В каталог nginx был скопирован конфигурационный файл веб-сервера, а в каталог librespeed/app — файлы backend-приложения Librespeed (Рисунок 2).

```
root@rekunid:~/project# cp /etc/nginx/sites-available/librespeed.conf ~/project/nginx
root@rekunid:~/project# cp -r /opt/librespeed/speedtest/* ~/project/librespeed/app/
root@rekunid:~/project# ls -la ~/project/nginx
total 12
drwxr-xr-x 2 root root 4096 Apr 29 13:22 .
drwxr-xr-x 7 root root 4096 Apr 29 13:22 ..
-rw-r--r-- 1 root root   0 Apr 29 13:22 Dockerfile
-rw-r--r-- 1 root root 2548 Apr 29 13:31 nginx.conf
root@rekunid:~/project# ls -la ~/project/librespeed/app
total 92
drwxr-xr-x  4 root root  4096 Apr 29 13:31 .
drwxr-xr-x  3 root root  4096 Apr 29 13:22 ..
-rw-r--r--  1 root root  1404 Apr 29 13:31 README.md
drwxr-xr-x 101 root root  4096 Apr 29 13:31 node_modules
-rw-r--r--  1 root root 39162 Apr 29 13:31 package-lock.json
-rw-r--r--  1 root root   492 Apr 29 13:31 package.json
drwxr-xr-x  3 root root  4096 Apr 29 13:31 src
-rw-r--r--  1 root root 26550 Apr 29 13:31 yarn.lock
```

Рисунок 2 — Проверка переноса конфигурации Nginx и файлов приложения Librespeed

В ходе выполнения работы был отредактирован конфигурационный файл веб-сервера Nginx. Вместо обслуживания статических файлов с использованием директив root и index, сервер был настроен на проксирование запросов к backend-сервисам.

В конфигурации был создан блок upstream, в котором описаны три сервиса librespeed-1, librespeed-2 и librespeed-3. Это позволяет реализовать балансировку нагрузки между несколькими контейнерами backend-

приложения. Далее в блоке `server` была настроена обработка входящих HTTP-запросов.

```
upstream librespeed_backend {
    server librespeed-1:8888;
    server librespeed-2:8888;
    server librespeed-3:8888;
}

server {
    listen 80;
    server_name _;

    server_tokens off;

    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header Referrer-Policy "strict-origin-when-cross-origin" always;
    add_header Content-Security-Policy "default-src 'self' 'unsafe-inline' 'unsafe-eval' data: blob:;" always;

    location / {
        proxy_pass http://librespeed_backend;
        proxy_http_version 1.1;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location /whoami {
        proxy_pass http://librespeed_backend/whoami;
        proxy_http_version 1.1;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

Рисунок 3 — Настройка проксирования и балансировки нагрузки в Nginx

На рисунке 4 представлен Dockerfile для backend-сервиса Librespeed. В нём используется образ `node:20-alpine`, задаётся рабочая директория, копируются файлы приложения и устанавливаются зависимости. Запуск сервиса выполняется командой `node src/SpeedTest.js`.

```
FROM node:20-alpine

WORKDIR /app

COPY app/package*.json ./
RUN npm install

COPY app/ ./

EXPOSE 8888

CMD ["node", "src/SpeedTest.js"]
```

Рисунок 4 — Dockerfile для сервиса Librespeed

На рисунке 5 представлен файл `.env` с переменными окружения. В нём указаны данные для подключения к PostgreSQL, а также названия volumes для хранения данных базы и логов Nginx.

```
GNU nano 7.2
POSTGRES_USER=admin
POSTGRES_PASSWORD=admin
POSTGRES_DB=speedtest

PG_DATA=pg_data
NGINX_LOGS=nginx_logs
```

Рисунок 5 — Настройка переменных окружения

На рисунке 6 представлен фрагмент файла `docker-compose.yml`. В нём описаны сервисы `nginx`, `librespeed-1`, `librespeed-2`, `librespeed-3` и `postgres`. Также указаны зависимости между контейнерами, `healthcheck` для backend-сервисов и `volumes` для хранения данных.

```
Version: '3.8'
services:
  nginx:
    build: ./nginx
    container_name: nginx
    ports:
      - "80:80"
    depends_on:
      librespeed-1:
        condition: service_started
      librespeed-2:
        condition: service_started
      librespeed-3:
        condition: service_started
    volumes:
      - nginx_logs:/var/log/nginx
      - /etc/letsencrypt:/etc/letsencrypt:ro
  librespeed-1:
    build: ./librespeed
    container_name: librespeed-1
    healthcheck:
      test: ["CMD", "node", "-e", "require('http').get('http://localhost:8888',()=>process.exit(0)).on('error',()=>process.exit(1))"]
      interval: 30s
      timeout: 10s
      retries: 3
  librespeed-2:
    build: ./librespeed
    container_name: librespeed-2
    healthcheck:
      test: ["CMD", "node", "-e", "require('http').get('http://localhost:8888',()=>process.exit(0)).on('error',()=>process.exit(1))"]
      interval: 30s
      timeout: 10s
      retries: 3
  librespeed-3:
    build: ./librespeed
    container_name: librespeed-3
    healthcheck:
      test: ["CMD", "node", "-e", "require('http').get('http://localhost:8888',()=>process.exit(0)).on('error',()=>process.exit(1))"]
      interval: 30s
      timeout: 10s
      retries: 3
  postgres:
    image: postgres:16-alpine
```

Рисунок 6 — Описание сервисов в Docker Compose

На рисунке 7 представлен фрагмент конфигурации Docker Compose. В нём показана настройка сервисов `nginx`, `librespeed-1`, `librespeed-2`, `librespeed-3` и `postgres`, а также указаны зависимости между контейнерами и проверка состояния backend-сервисов.

```

version: '3.8'
services:
  nginx:
    build: ./nginx
    container_name: nginx
    ports:
      - "80:80"
    depends_on:
      librespeed-1:
        condition: service_started
      librespeed-2:
        condition: service_started
      librespeed-3:
        condition: service_started
    volumes:
      - nginx_logs:/var/log/nginx
      - /etc/letsencrypt:/etc/letsencrypt:ro
  librespeed-1:
    build: ./librespeed
    container_name: librespeed-1
    healthcheck:
      test: ["CMD", "wget", "--spider", "http://localhost:8888"]
      interval: 30s
      timeout: 10s
      retries: 3
  librespeed-2:
    build: ./librespeed
    container_name: librespeed-2
    healthcheck:
      test: ["CMD", "wget", "--spider", "http://localhost:8888"]
      interval: 30s
      timeout: 10s
      retries: 3
  librespeed-3:
    build: ./librespeed
    container_name: librespeed-3
    healthcheck:
      test: ["CMD", "wget", "--spider", "http://localhost:8888"]
      interval: 30s
      timeout: 10s
      retries: 3
  postgres:
    image: postgres:16-alpine

```

Рисунок 7 — Фрагмент конфигурации Docker Compose

Далее была выполнена установка docker.io и docker-compose-v2. После установки были проверены версии Docker и Docker Compose (Рисунок 8, Рисунок 9).

```

root@rekunid:~/project# apt update
apt install docker.io docker-compose-v2 -y
Hit:1 http://mirror.selectel.ru/ubuntu noble InRelease
Get:2 http://mirror.selectel.ru/ubuntu noble-updates InRelease [126 kB]
Get:3 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:4 http://mirror.selectel.ru/ubuntu noble-backports InRelease [126 kB]
Hit:5 https://mirror.selectel.ru/3rd-party/cloud-init-deb/noble noble InRelease

```

Рисунок 8 — Установка Docker и Docker Compose

```

Docker version 29.1.3, build 29.1.3-0ubuntu3~24.04.2
Docker Compose version 2.40.3+ds1-0ubuntu1~24.04.1
root@rekunid:~/project#

```

Рисунок 9 — Проверка версий Docker и Docker Compose

После проверки конфигурации была выполнена сборка Docker-образов. На рисунке 10 показано, что образы для сервисов librespeed-1, librespeed-2, librespeed-3 и nginx были успешно собраны.

```
=> [nginx] resolving provenance
[+] Building 4/4
✓ librespeed-1 Built
✓ librespeed-2 Built
✓ librespeed-3 Built
✓ nginx Built
root@rekunid:~/project#
```

Рисунок 10 — Сборка Docker-образов сервисов

Далее был выполнен запуск проекта командой `docker compose up -d`. Docker Compose автоматически создал сеть `project_default`, volumes `project_pg_data` и `project_nginx_logs`, а также запустил контейнеры приложения (Рисунок 11).

```
root@rekunid:~/project# docker compose up -d
WARN[0000] /root/project/docker-compose.yml: the
onfusion
[+] Running 13/13
✓ postgres Pulled
  ✓ 20f9cf2e9893 Pull complete
  ✓ 420ca0de84ca Pull complete
  ✓ 3b7e6bf074f6 Pull complete
  ✓ 72393ada9150 Pull complete
  ✓ c740b6b7dd0b Pull complete
  ✓ 0d3e610f9e0f Pull complete
  ✓ d9681cd68a94 Pull complete
  ✓ 282a6867e326 Pull complete
  ✓ abdc7c6150b5 Pull complete
  ✓ 5ef55a6c860c Pull complete
  ✓ fb2a3b9ecbd5 Download complete
  ✓ 9827fda4490e Download complete
[+] Running 8/8
✓ Network project_default Created
✓ Volume project_pg_data Created
✓ Volume project_nginx_logs Created
✓ Container librespeed-2 Started
✓ Container postgres Started
✓ Container librespeed-3 Started
✓ Container librespeed-1 Started
✓ Container nginx Started
root@rekunid:~/project#
```

Рисунок 11 — Запуск контейнеров через Docker Compose

После проверки конфигурации был просмотрен итоговый вариант файла `docker-compose.yml`. На рисунке 12 показано, что для сервисов librespeed-1,

librespeed-2 и librespeed-3 настроены параметры сборки, имена контейнеров, проверка состояния через healthcheck и подключение к общей сети.

```
onfusion
name: project
services:
  librespeed-1:
    build:
      context: /root/project/librespeed
      dockerfile: Dockerfile
    container_name: librespeed-1
    healthcheck:
      test:
        - CMD
        - wget
        - --spider
        - http://localhost:3000
      timeout: 10s
      interval: 30s
      retries: 3
    networks:
      default: null
  librespeed-2:
    build:
      context: /root/project/librespeed
      dockerfile: Dockerfile
    container_name: librespeed-2
    healthcheck:
      test:
        - CMD
        - wget
        - --spider
        - http://localhost:3000
      timeout: 10s
      interval: 30s
      retries: 3
    networks:
      default: null
  librespeed-3:
    build:
      context: /root/project/librespeed
      dockerfile: Dockerfile
    container_name: librespeed-3
    healthcheck:
      test:
        - CMD
        - wget
        - --spider
        - http://localhost:3000
      timeout: 10s
      interval: 30s
      retries: 3
    networks:
      default: null
root@rekunid:~/project#
```

Рисунок 12 — Проверка итоговой конфигурации сервисов Librespeed

Далее была выполнена проверка запущенных контейнеров. На рисунке 13 видно, что контейнеры nginx, postgres и все три сервиса librespeed находятся в состоянии Up, при этом для PostgreSQL и backend-сервисов отображается статус healthy.

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
4ea1e0cb212f	project-nginx	"/docker-entrypoint..."	45 seconds ago	Up 40 seconds	0.0.0.0:80->80/tcp,
cp_nginx					
5ddb4508d8b	postgres:16-alpine	"docker-entrypoint.s..."	52 seconds ago	Up 42 seconds (healthy)	5432/tcp
71712db7fa57	project-librespeed-3	"docker-entrypoint.s..."	52 seconds ago	Up 41 seconds (healthy)	8888/tcp
librespeed-3					
cef4c3eda10b	project-librespeed-2	"docker-entrypoint.s..."	52 seconds ago	Up 42 seconds (healthy)	8888/tcp
librespeed-2					
4353ebdda47b	project-librespeed-1	"docker-entrypoint.s..."	52 seconds ago	Up 41 seconds (healthy)	8888/tcp
librespeed-1					

Рисунок 13 — Проверка состояния запущенных контейнеров

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы была настроена многоконтейнерная инфраструктура с использованием Docker Compose. Были подготовлены конфигурационные файлы для сервисов nginx, libspeed и postgres, а также настроены переменные окружения и volumes для хранения данных.

Также был настроен Nginx как обратный прокси-сервер, который перенаправляет запросы на несколько backend-сервисов Libspeed. За счёт этого была реализована простая балансировка нагрузки между контейнерами.

После запуска проекта была выполнена проверка работы контейнеров с помощью команд Docker. В результате все основные сервисы были успешно запущены, а для части контейнеров отобразился статус healthy, что подтверждает корректную работу инфраструктуры.

Таким образом, в ходе работы были получены практические навыки настройки Docker Compose, запуска нескольких связанных сервисов, использования volumes и проверки состояния контейнеров в Linux.